

Anexo 10 — Runtime de evaluacion combinado head+brazo

Archivo: Scripts/run_combined_head_arm.py

Para que sirvio:

Corre la cabeza y el brazo juntos en una misma simulacion para validar que ambas policias coexisten, generando video de varias configuraciones de prueba.

```
1 | """Runtime combinado: v13 (head tracking) + v7e (arm grab) en una sola sim.
2 |
3 | La cabeza apunta a la BOLA AMARILLA (en lugar del target rojo mocap). El brazo
4 | trata de agarrarla. Ambas policias leen sus obs de la MISMA MjData compartida.
5 |
6 | Genera un video MP4 mostrando el comportamiento con varias alturas de spawn
7 | para ver cuanto generaliza v7e fuera de su distribucion de training (offset
8 | maximo en training fue 17cm, aca lo subimos progresivamente).
9 |
10 | Uso:
11 |     python Scripts/run_combined_head_arm.py [--episodes 6] [--video out.mp4]
12 |
13 | import argparse
14 | import sys
15 | from pathlib import Path
16 |
17 | SCRIPTS_DIR = Path(__file__).resolve().parent
18 | sys.path.insert(0, str(SCRIPTS_DIR))
19 |
20 | import numpy as np
21 | import mujoco
22 | from PIL import Image, ImageDraw, ImageFont
23 | from stable_baselines3 import PPO
24 | from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize
25 |
26 | from rl_env import DUMHeadTrackingEnv
27 | from rl.envs.grab_env import DUMGrabEnv
28 |
29 |
30 | class CombinedHeadArmEnv(DUMGrabEnv):
31 |     """DUMGrabEnv extendido: en cada step, ANTES de aplicar el ctrl del brazo,
32 |     computa la obs del head (con target = yellow_ball), pasa por VecNormalize de v13,
33 |     predice la accion del head y mete su ctrl en los actuadores correspondientes.
34 |
35 |     El target_world del head policy es la posicion de la bola amarilla.
36 |     """
37 |
38 |     def __init__(self, head_model, head_vn, **kwargs):
39 |         super().__init__(**kwargs)
40 |         self.head_model = head_model
41 |         self.head_vn = head_vn
42 |
43 |         # IDs/addrds para computar obs del head desde la MjData del grab env
44 |         self._head_body_id = mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_BODY, 'LenteExt_link')
45 |         self._lens_site_id = mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_SITE, 'lens_center')
46 |         head_jnames = ['Neck_joint', 'HeadBase_joint', 'HeadRotation_joint']
47 |         head_jids = [mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_JOINT, n) for n in head_jnames]
48 |         self._head_qpos_adr_c = np.array([self.model.jnt_qposadr[j] for j in head_jids])
49 |         self._head_dof_adr_c = np.array([self.model.jnt_dofadr[j] for j in head_jids])
50 |         hb_jid = mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_JOINT, 'HipBody_joint')
51 |         self._hb_qpos_adr_c = self.model.jnt_qposadr[hb_jid]
52 |         self._hb_dof_adr_c = self.model.jnt_dofadr[hb_jid]
53 |         head_anames = ['act_Neck', 'act_HeadBase', 'act_HeadRot']
54 |         self._head_act_ids_c = [mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_ACTUATOR, n)
55 |                                for n in head_anames]
56 |         self._head_ctrl_lo_c = np.array([self.model.actuator_ctrlrange[a, 0] for a in self._head_act_ids_c])
57 |         self._head_ctrl_hi_c = np.array([self.model.actuator_ctrlrange[a, 1] for a in self._head_act_ids_c])
58 |
59 |         # Estado para tracking de velocidad del target (bola)
60 |         self._target_world_prev = None
61 |         self._prev_head_action = np.zeros(3, dtype=np.float32)
62 |         # Datos del HEAD policy para la obs (los necesita pero no los chequea)
63 |         self._head_theta_log = 0.0 # para visualizar
64 |
65 |     def _compute_head_obs(self):
66 |         """Replica DUMHeadTrackingEnv._get_obs operando sobre nuestra MjData."""
67 |         ball_pos = self._ball_pos_world()
68 |         target_world = ball_pos
69 |         if self._target_world_prev is None:
70 |             self._target_world_prev = target_world.copy()
71 |         dt_ctrl = self.frame_skip * float(self.model.opt.timestep)
72 |         head_mat = self.data.xmat[self._head_body_id].reshape(3, 3)
73 |         lens_origin = self.data.site_xpos[self._lens_site_id]
74 |         tloc = head_mat.T @ (target_world - lens_origin)
```

```

75 |         tdist = float(np.linalg.norm(tloc))
76 |         tdir = tloc / (tdist + 1e-9)
77 |         tvel_world = (target_world - self._target_world_prev) / dt_ctrl
78 |         tvel = head_mat.T @ tvel_world
79 |
80 |         qpos_head = self.data.qpos[self._head_qpos_adr_c].copy()
81 |         qvel_head = self.data.qvel[self._head_dof_adr_c].copy()
82 |         qpos_hb = float(self.data.qpos[self._hb_qpos_adr_c])
83 |         qvel_hb = float(self.data.qvel[self._hb_dof_adr_c])
84 |
85 |         obs = np.concatenate([
86 |             qpos_head, qvel_head,          # 6
87 |             [qpos_hb, qvel_hb],          # 2
88 |             tdir, [tdist],                # 4
89 |             tvel,                        # 3
90 |             self._prev_head_action.astype(np.float64), # 3
91 |         ])
92 |         # Para overlay del video – angulo entre head forward y direccion a target
93 |         # HEAD_FORWARD_LOCAL = (0, -1, 0) en frame local del LenteExt_link
94 |         head_forward_local = np.array([0.0, -1.0, 0.0])
95 |         forward_world = head_mat @ head_forward_local
96 |         if tdist > 1e-6:
97 |             delta = (target_world - lens_origin) / tdist
98 |             cos_a = float(np.clip(np.dot(forward_world, delta), -1.0, 1.0))
99 |             self._head_theta_log = float(np.arccos(cos_a))
100 |         return obs
101 |
102 | def _apply_action(self, arm_action):
103 |     # Ctrl del brazo (parent class)
104 |     full_ctrl = super()._apply_action(arm_action)
105 |     # Obs del head -> (opcionalmente normalizada) -> action
106 |     head_obs = self._compute_head_obs()
107 |     if self.head_vn is not None:
108 |         head_obs_input = self.head_vn.normalize_obs(head_obs[None, :])
109 |     else:
110 |         head_obs_input = head_obs[None, :]
111 |     head_action, _ = self.head_model.predict(head_obs_input, deterministic=True)
112 |     head_action = head_action[0]
113 |     # Mapear [-1,1] a ctrlrange
114 |     a = np.clip(head_action, -1.0, 1.0).astype(np.float64)
115 |     head_ctrl = (a + 1.0) * 0.5 * (self._head_ctrl_hi_c - self._head_ctrl_lo_c) + self._head_ctrl_lo_c
116 |     for i, aid in enumerate(self._head_act_ids_c):
117 |         full_ctrl[aid] = head_ctrl[i]
118 |     # Guardar prev_action y prev_target
119 |     self._prev_head_action = head_action.astype(np.float32).copy()
120 |     self._target_world_prev = self._ball_pos_world().copy()
121 |     return full_ctrl
122 |
123 |
124 | def main():
125 |     p = argparse.ArgumentParser()
126 |     p.add_argument("--episodes", type=int, default=6)
127 |     p.add_argument("--video", type=str, default="runs/grab_phasel_v7e/combined_head_arm.mp4")
128 |     p.add_argument("--head-policy", type=str, default="runs/ppo_dum_v13/final.zip")
129 |     p.add_argument("--arm-policy", type=str, default="runs/grab_phasel_v7e/final.zip")
130 |     p.add_argument("--enable-throw", action="store_true",
131 |                     help="Activar HELD->THROWN post-grab (v9)")
132 |     args = p.parse_args()
133 |
134 |     # Cargar HEAD policy (v13 fue entrenado sin VecNormalize, asi que pasamos obs cruda)
135 |     print("[setup] Cargando head policy v13...")
136 |     head_vn_path = Path(args.head_policy).parent / "vecnormalize.pkl"
137 |     head_vn = None
138 |     if head_vn_path.exists():
139 |         dummy_head = DummyVecEnv([lambda: DUMHeadTrackingEnv()])
140 |         head_vn = VecNormalize.load(str(head_vn_path), dummy_head)
141 |         head_vn.training = False
142 |         head_vn.norm_reward = False
143 |         print(f"[setup] head VecNormalize cargado")
144 |     else:
145 |         print(f"[setup] sin VecNormalize del head (no encontrado en {head_vn_path})")
146 |     head_model = PPO.load(args.head_policy)
147 |
148 |     # Cargar VecNormalize del ARM
149 |     print("[setup] Cargando arm policy v7e...")
150 |     dummy_arm = DummyVecEnv([lambda: CombinedHeadArmEnv(head_model, head_vn, phase=1)])
151 |     arm_vn_path = Path(args.arm_policy).parent / "vecnormalize.pkl"
152 |     arm_vn = VecNormalize.load(str(arm_vn_path), dummy_arm)
153 |     arm_vn.training = False
154 |     arm_vn.norm_reward = False
155 |     arm_model = PPO.load(args.arm_policy, env=arm_vn)
156 |
157 |     raw = arm_vn.envs[0]
158 |     print(f"[setup] env OK. nu={raw.model.nu} dof={raw.model.nv}")
159 |     if args.enable_throw:

```

```

160 |         raw.set_throw_enabled(True)
161 |         print("[setup] throw ENABLED (HELD->THROWN post-grab)")
162 |
163 |     # Renderer offscreen
164 |     renderer = mujoco.Renderer(raw.model, height=480, width=480)
165 |     try:
166 |         font = ImageFont.truetype("arial.ttf", 13)
167 |         font_small = ImageFont.truetype("arial.ttf", 11)
168 |     except Exception:
169 |         font = ImageFont.load_default()
170 |         font_small = font
171 |
172 |     # v16: cap 35/15. Solo falling (no static), distribucion de training.
173 |     configs = [
174 |         (0.00, 0.00, 0.0, "rehearsal off=0"),
175 |         (0.15, 0.05, -2.5, "falling small (15, 5)"),
176 |         (0.20, 0.08, -2.5, "falling small (20, 8)"),
177 |         (0.25, 0.10, -2.5, "falling mid (25, 10)"),
178 |         (0.30, 0.12, -2.5, "falling high (30, 12)"),
179 |         (0.35, 0.10, -2.5, "falling MAX-up (35, 10)"),
180 |         (0.30, 0.15, -2.5, "falling MAX-out (30, 15)"),
181 |         (0.35, 0.15, -2.5, "falling MAX (35, 15) #1"),
182 |         (0.35, 0.15, -2.5, "falling MAX (35, 15) #2"),
183 |         (0.35, 0.15, -2.5, "falling MAX (35, 15) #3"),
184 |         (0.33, 0.14, -2.5, "falling near-max (33, 14)"),
185 |         (0.30, 0.10, -2.5, "falling mid-confort (30, 10)"),
186 |     ]
187 |
188 |     if len(configs) < args.episodes:
189 |         configs = configs + [configs[-1]] * (args.episodes - len(configs))
190 |     configs = configs[:args.episodes]
191 |
192 |     frames = []
193 |     grabs = 0
194 |     rews = []
195 |     lens = []
196 |     head_thetas = []
197 |     print(f"\n{'ep':>3s} {'config':>50s} {'rew':>8s} {'len':>4s} {'grab':>5s} {'min_d':>6s} {'head_deg(°)':>10s}")
198 |     print("-" * 100)
199 |     for ep, (up_off, out_off, gravity, label) in enumerate(configs):
200 |         # Resetear
201 |         raw.set_curriculum_max_offsets(0.0, 0.0)
202 |         raw.set_falling_active(False)
203 |         obs = arm_vn.reset()
204 |         # Forzar spawn segun config
205 |         raw.model.opt.gravity[:] = (0.0, 0.0, gravity)
206 |         ee = raw.data.site_xpos[raw._ee_site_id]
207 |         sp = np.array([ee[0], ee[1] - out_off, ee[2] + 0.03 + up_off])
208 |         raw.data.qpos[raw._ball_qpos_adr:raw._ball_qpos_adr+3] = sp
209 |         raw.data.qvel[raw._ball_dof_adr:raw._ball_dof_adr+6] = 0.0
210 |         mujoco.mj_forward(raw.model, raw.data)
211 |         raw._ball_spawn_pos = sp
212 |         raw._curriculum_up_offset = up_off
213 |         raw._curriculum_out_offset = out_off
214 |         raw._target_world_prev = sp.copy()
215 |         raw._prev_head_action = np.zeros(3, dtype=np.float32)
216 |
217 |         done = False
218 |         ep_rew = 0.0
219 |         ep_len = 0
220 |         grabbed = False
221 |         min_dist = 1.0
222 |         theta_acc = []
223 |         while not done:
224 |             action, _ = arm_model.predict(obs, deterministic=True)
225 |             obs, r, done, infos = arm_vn.step(action)
226 |             info = infos[0]
227 |             ep_rew += float(r[0])
228 |             ep_len += 1
229 |             min_dist = min(min_dist, info['ee_ball_dist'])
230 |             if info.get('grabbed_now', False):
231 |                 grabbed = True
232 |                 theta_acc.append(np.rad2deg(raw._head_theta_log))
233 |
234 |         # Render
235 |         renderer.update_scene(raw.data, camera=-1)
236 |         img = Image.fromarray(renderer.render())
237 |         d = ImageDraw.Draw(img)
238 |         ball_z = raw.data.qpos[raw._ball_qpos_adr+2]
239 |         for i, txt in enumerate([
240 |             "v13 head + v7e arm (combined)",
241 |             label,
242 |             f"ep {ep} step {ep_len:3d} arm={raw._side}",
243 |             f"ball_z {ball_z*100:.1f}cm",
244 |             f"dist {info['ee_ball_dist']*100:.2f}cm",

```

```

245 |         f"lever {info['lever_q']*1000:.2f}mm",
246 |         f"head_deg {np.rad2deg(raw._head_theta_log):5.1f}°",
247 |         f"rew {ep_rew:+.1f}",
248 |         "GRAB!" if grabbed else "...",
249 |     ]):
250 |         color = (100, 255, 100) if grabbed else (255, 240, 100)
251 |         d.text((6, 6+i*14), txt, fill=color, font=font_small)
252 |     frames.append(np.array(img))
253 |
254 |     done = bool(dones[0])
255 |
256 |     rews.append(ep_rew)
257 |     lens.append(ep_len)
258 |     head_thetas.append(np.mean(theta_acc) if theta_acc else 0.0)
259 |     if grabbed:
260 |         grabs += 1
261 |     print(f"{ep:>3d} {label[:50]:>50s} {ep_rew:+8.1f} {ep_len:>4d} {'YES' if grabbed else 'no':>5s} "
262 |           f"{min_dist*100:>6.2f} {np.mean(theta_acc) if theta_acc else 0.0:>10.1f}",
263 |           flush=True)
264 |
265 |     print()
266 |     print(f"TOTAL: grabs={grabs}/{args.episodes}  ep_rew={np.mean(rews):+.1f} "
267 |           f"ep_len={np.mean(lens):.1f}  avg_head_deg={np.mean(head_thetas):.1f}°")
268 |
269 |     # Save video
270 |     import imageio
271 |     out_path = Path(args.video)
272 |     out_path.parent.mkdir(parents=True, exist_ok=True)
273 |     imageio.mimsave(str(out_path), frames, fps=50, quality=8)
274 |     print(f"\nVideo: {out_path} ({len(frames)} frames @ 50fps)")
275 |
276 |
277 | if __name__ == "__main__":
278 |     main()
279 |

```